

Base de datos de Territorio Preparado: informe de montaje en Supabase

Proyecto: Territorio Preparado, RETO-01 Cali Activa (Hackathon Smart City Expo Cali 2026).

Fecha: 4 de octubre de 2026. **Quién lo hizo:** William Ortiz, con Claude Code.

Estado: la base está **montada en Supabase dev**: 12 migraciones aplicadas y la API expone solo `api`, verificado contra el proyecto real. Faltan el MFA, el hook del token y la carga de los datos reales (sección 8).

Este documento no contiene contraseñas, claves ni datos personales.

1. Resumen

Qué	Resultado
Modelo de datos (Hito 1)	6 esquemas, 31 tablas, 24 vistas, 32 políticas de seguridad por fila (RLS) y 35 permisos por rol
SQL (Hito 1)	12 migraciones en <code>db/supabase/migrations/</code> , con el formato de la CLI de Supabase
Datos reales (Hito 2)	Script de carga con verificación sha256: 22 comunas, 342 barrios, 2.991 espacios, 660 zonas de amenaza y 182 juntas de acción comunal
Pruebas	69 pruebas del SQL y 17 de la carga, todas en verde (<code>cd db && npm test</code>)
Repositorio	Pasado a privado ; 3 integrantes invitados con permiso de escritura; pull request #2 abierto hacia main
Supabase	Proyecto dev creado (ref <code>rqxltixtsiqsaxpdue</code>), CLI enlazada, 12 de 12 migraciones aplicadas

2. Qué se hizo, paso a paso

- SQL del Hito 1.** Se tradujo el modelo de `db/docs/` a 11 migraciones: esquemas, tablas con sus reglas de integridad, auditoría encadenada, catálogos, permisos, RLS, vistas y funciones de la API, y privilegios. Se probaron en PGlite, que es PostgreSQL 18 con PostGIS 3.6 corriendo en Node, porque en el equipo no hay Docker.
- Carga de datos reales (Hito 2).** Se escribió `db/scripts/cargar_datos_reales.mjs` y la migración 12 (`id_fuente` para recargar las juntas sin duplicarlas). Se comprobó que los cruces de amenaza calculados por PostGIS coinciden **uno a uno** con los que precalculó la app: 884 espacios con cruce de inundación y 1.369 con cruce sísmico.
- Repositorio privado.** El repo `leonidas452528/hackaton-camaracomercio2026` pasó a privado. La demo de GitHub Pages sigue publicada, porque la cuenta es GitHub Pro.
- Integrantes.** Se invitó con permiso **Write** a `helynecheverry` (Herlin), `PabloEARangoM` (Pablo) y `bryanmavi` (Bryan). Bryan ya aceptó; Herlin y Pablo, todavía no.
- Pull request.** La rama `db/esquema-inicial` se subió y quedó como PR #2 hacia main, para que la

revise el responsable de la base de datos.

- 6. **Proyecto en Supabase.** Se creó el proyecto dev con la cuenta de GitHub. Ref: `rqxltixtsiqsaxapdue`; URL de la API: `https://rqxltixtsiqsaxapdue.supabase.co`.
- 7. **CLI de Supabase.** Se instaló como dependencia de desarrollo de db/ (versión 2.119.0), así todo el equipo usa la misma versión. Se generó `db/supabase/config.toml`, con `api` como único esquema expuesto, el hook del token, MFA por aplicación (TOTP) y `sin seed.sql`.
- 8. **Enlace.** `npx supabase login` y `npx supabase link --project-ref rqxltixtsiqsaxapdue` se corrieron en una terminal normal. Dentro de Claude Code el login falla porque no hay una terminal interactiva.
- 9. **Migraciones.** `npx supabase db push --dry-run` mostró las 12 migraciones pendientes y nada más. `npx supabase db push` las aplicó sin errores, y `npx supabase migration list` confirma que lo local y lo remoto coinciden.
- 10. **Exposición de la API.** La primera consulta a la API respondió `PGRST106` (esquema no expuesto). William dejó `api` como esquema expuesto en *Project Settings* → *Data API*, y se repitió la verificación como público (`anon`), con la clave pública:

Consulta como público (anon)	Resultado en Supabase
<code>api.amenazas, servicios, funciones_espacio, entidades, responsabilidades, parametros_reglas</code>	5, 12, 7, 24, 4 y 5 filas: los catálogos sembrados
<code>api.espacios</code>	0 filas: falta la carga de datos reales
<code>api.decisiones</code> y <code>api.auditoria</code>	Bloqueado (42501): solo con sesión y rol
Función de escritura <code>registrar_medicion</code>	Bloqueada (42501): el público no escribe
Esquemas <code>ops</code> y <code>public</code>	No expuestos (PGRST106)

3. Qué quedó en la base

Esquema	Contenido	¿Lo ve la API?
<code>ref</code>	9 tablas de catálogos: 5 amenazas, 12 servicios y tareas, 7 funciones del espacio, 24 entidades, 5 parámetros de planeación, 4 filas de la matriz de responsabilidades y un protocolo de 11 pasos	No
<code>geo</code>	6 tablas de territorio (comunidades, barrios, espacios, zonas de amenaza, mediciones, organizaciones comunitarias) y 2 vistas (exposición y mediciones vigentes)	No
<code>ops</code>	9 tablas de operación (recomendaciones, decisiones, brechas, seguimientos, tareas, reportes comunitarios, retornos, lecturas IoT simuladas, notificaciones simuladas)	No
<code>idn</code>	5 tablas de identidad (perfiles, cargos, asignaciones, permisos por rol, autorizaciones de tratamiento)	No
<code>aud</code>	Eventos de auditoría y verificaciones de la cadena	No
<code>api</code>	22 vistas y 12 funciones: el único contrato con el frontend (<code>db/docs/CONTRATO_API.md</code>)	Sí

Todo lo institucional (entidades, matriz de responsabilidades, protocolo, funciones del espacio por amenaza) está marcado como **propuesta**: no lo ha validado la Secretaría de Gestión del Riesgo. No hay plazos ni cifras sin fuente. Los parámetros de baños, agua y superficie salen del Manual Esfera 2018.

4. Reglas que la base hace cumplir

- **La recomendación no reemplaza a la autoridad.** Solo `gestion_riesgo`, con verificación en dos pasos, registra una activación, y debe citar el acto administrativo. Si no sigue la recomendación, debe justificar.
- **No hay datos de personas.** Ninguna tabla guarda damnificados. Los reportes son conteos agregados, y los textos libres rechazan correos, teléfonos y números largos.
- **Desconocido no es cero.** Sin una medición verificada, el faltante queda vacío. Verificar exige evidencia y una persona distinta de quien reportó.
- **Cada quien ve lo suyo.** Las entidades ven sus brechas y tareas; las juntas, su zona; la consulta, solo agregados, con los conteos menores a 5 ocultos.
- **Cambios de cuenta inmediatos.** El rol se lee de la base en cada consulta, así que una suspensión o un traspaso de cargo surte efecto al instante.
- **Auditoría verificable.** Los eventos se encadenan con SHA-256 y no se pueden borrar ni modificar. Las pruebas incluyen una manipulación deliberada, que la verificación detecta.
- **Una sola puerta.** Ningún rol escribe directamente en las tablas: todo pasa por las funciones de `api`, que verifican permiso, alcance y MFA.

5. Verificaciones realizadas

Prueba	Resultado
69 pruebas de seguridad y flujo (PGLite)	Todas pasan
17 pruebas de carga con los datos reales (PGLite)	Todas pasan; la carga se puede repetir y se deshace completa ante una falla
Cruces de PostGIS frente a los de la app	0 diferencias en 2.991 espacios
Huellas sha256 de los 9 archivos crudos de la IDESC	Coinciden con <code>manifest.json</code>
<code>supabase db push --dry-run</code>	Solo las 12 migraciones esperadas
<code>supabase db push</code>	12 de 12 aplicadas sin errores
<code>supabase migration list</code>	Lo local y lo remoto coinciden
API REST de Supabase como anon	Catálogos legibles; decisiones, auditoría y escrituras bloqueadas; ops y public no expuestos

Tiempos medidos en PGLite: una ficha, 2 ms; la lista de 2.991 espacios, 36 ms; todas las fichas completas, alrededor de 1 s.

6. Archivos en el repositorio

Ruta	Qué es
db/supabase/migrations/	Las 12 migraciones (nunca se edita una ya aplicada: los cambios van en una migración nueva)
db/supabase/config.toml	Configuración de la CLI (sin secretos)
db/scripts/ cargar_datos_reales.mjs	Carga del Hito 2
db/tests/	Imitación mínima de Supabase para las pruebas, ejecutor y pruebas
db/docs/	Modelo, diccionario, roles, decisiones, contrato de la API, este informe y la guía de acceso del equipo
docs/cumplimiento/	Normas verificadas, entes decisores, red comunitaria y matriz normativa

7. Lo que NO está en el repositorio (y no debe estar)

La contraseña de la base de datos, el token de la CLI, la clave `service_role` o secreta, y las URL de conexión. El token de la CLI quedó solo en el equipo de William; la contraseña, en su gestor de contraseñas.

8. Pendientes para terminar el montaje

1. ☒ **Exponer solo api:** hecho y verificado (public responde "no expuesto").
2. ☐ **Activar el MFA TOTP:** *Authentication → Multi-Factor*.
3. ☐ **Activar el hook del token:** *Authentication → Hooks → Customize Access Token (JWT) Claims*, tipo Postgres, esquema idn, función `custom_access_token_hook`.
4. ☐ **Cargar los datos reales:** con la cadena del *Session pooler* (botón **Connect**), solo en la terminal:

```
cd ~/hackathon-cali-2026/db
DATABASE_URL='postgresql://postgres.rqxltixtsiqsaxapdue:CONTRASEÑA@HOST-DEL-POOLER:5432/postgres?
sslmode=require' npm run cargar
```

5. ☐ **Invitar al equipo en Supabase:** *Organization settings → Team → Invite member*, con el rol **Developer** (guía aparte: `ACCESO_EQUIPO`).
6. ☐ **Revisar y fusionar el PR #2.** Lo hace el responsable de la base de datos, que además responde las 10 decisiones abiertas de `MODELO_DATOS.md` §9.
7. ☐ **Hito 5:** crear las 13 cuentas de demostración por rol, para probar la app con cada perfil.

9. Decisiones tomadas en esta sesión

- **Las pruebas corren en PGLite**, sin Docker ni Supabase local: cualquier integrante las corre con `npm test`.
- **Los catálogos van en migraciones**, no en `seed.sql`, porque el proyecto se arma solo con

migraciones (ADR-0009).

- **La autorización lee la base, no el token**, para que la suspensión sea inmediata.
- **Se cargan las capas derivadas de la app**, no las crudas. Así los cruces son idénticos: la capa sísmica de la app usa 5 de los 16 polígonos de microzonificación, los que tienen susceptibilidad a licuación o corrimiento.
- **Los datos se cargan tal como vienen**. No se inventa ni se corrige en silencio: 52 espacios sin comuna, 23 juntas con un código de barrio que no está en la capa, y 3 huellas y 18 zonas con geometría no válida se cargan así y quedan reportadas.
- **La conexión verifica el certificado TLS**: no se desactiva la verificación.